

Math Nuggets for GEN AI

Positional Embeddings: Teaching AI Where Words Are

By Dr. Anish Roychowdhury, Plaksha University

September 21, 2025

Why Position Matters

Consider these sentences:

- "Alice loves Bob"
- "Bob loves Alice"

Same words, completely different meanings! While humans naturally understand word order, neural networks initially see words as an unordered collection. Positional embeddings solve this by giving each word position a unique mathematical "fingerprint."

Where Do Positional Embeddings Fit?

In the transformer architecture, positional embeddings are added at a very specific point in the processing pipeline. After text is converted into tokens and then into word embeddings (which capture semantic meaning), positional embeddings are computed separately and then simply added to the word embeddings. This happens before any attention layers process the data.

The beauty of this approach is its simplicity: rather than fundamentally changing the architecture, we enhance existing word embeddings with position information. This means each word gets both its original meaning and knowledge of where it sits in the sentence - enabling the model to understand both "what" and "where" simultaneously.

The Transformer Pipeline

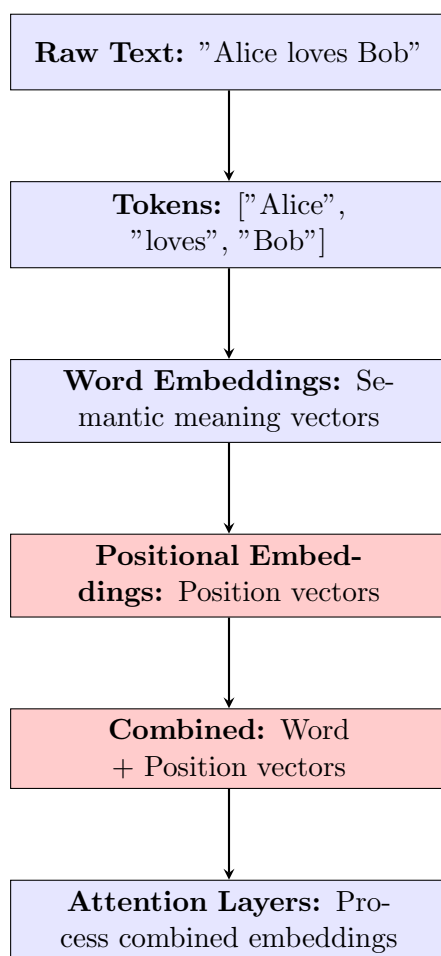


Figure 1: Simplified transformer pipeline with positional embeddings highlighted

The Mathematical Solution

The transformer uses sinusoidal functions to create unique positional patterns:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right) \quad (\text{Even dimensions}) \quad (1)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right) \quad (\text{Odd dimensions}) \quad (2)$$

Where:

- pos = position in sequence (0, 1, 2, ...)
- i = dimension index (0, 1, 2, ...)
- d = embedding dimension (typically 512)

How Sinusoidal Encoding Works

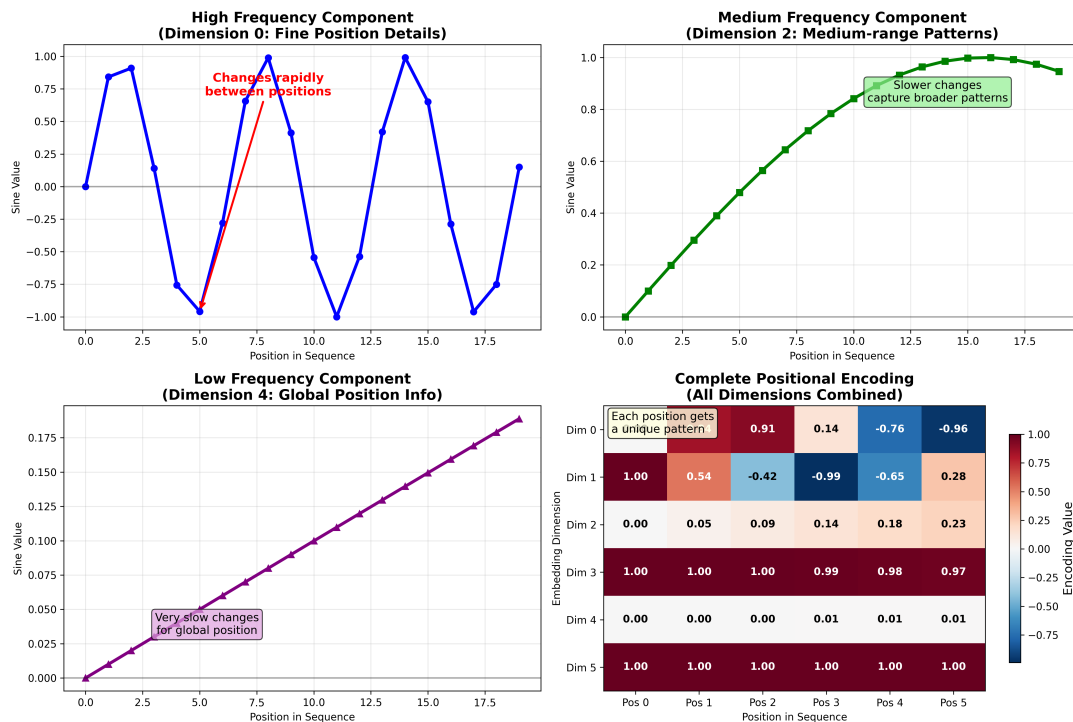


Figure 2: Sinusoidal encoding creates unique patterns for each position

Key Properties:

- **Unique fingerprints:** Each position gets a distinct pattern
- **Smooth transitions:** Adjacent positions have similar patterns
- **Different frequencies:** Fast changes for fine details, slow changes for broad patterns
- **Bounded values:** All values stay between -1 and 1

Worked Example: "Alice loves Bob"

Let's compute positional embeddings with simplified parameters:

- Embedding dimension: $d = 6$
- Sequence: ["Alice" (pos=0), "loves" (pos=1), "Bob" (pos=2)]

The figures above demonstrate the complete process of combining word and positional embeddings. Figure 1 shows how adding positional information changes the magnitude of embedding vectors - each word retains its semantic meaning while gaining position awareness. Figure 2 provides a detailed view of the addition process: word embeddings (top section) contain semantic information, positional embeddings (middle section) contain unique position patterns, and the combined result (bottom section) preserves both types of information. Notice how each position creates a distinct pattern that can be distinguished by the attention mechanism.

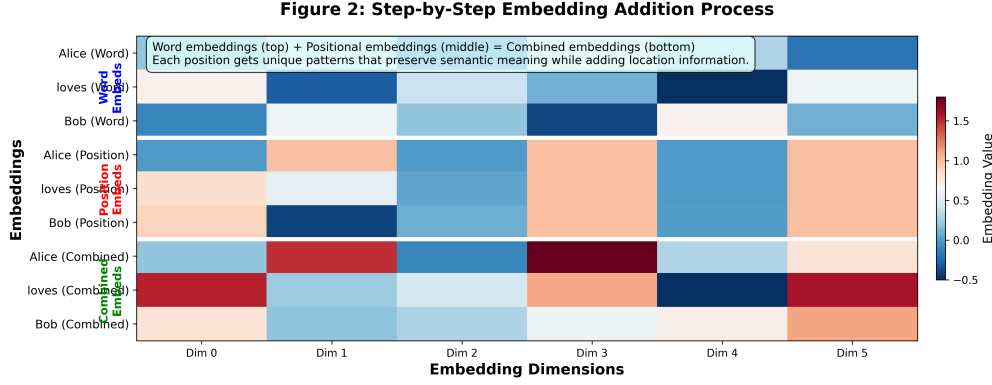
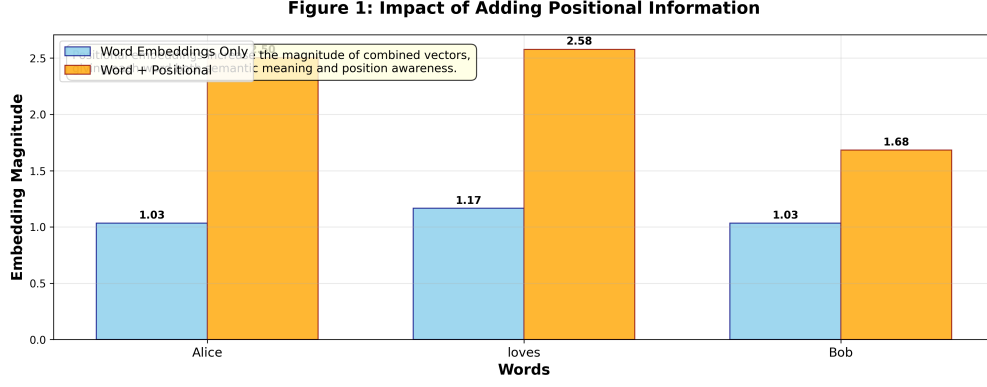


Figure 3: Complete worked example showing the embedding combination process

Step 1: Calculate Positional Embeddings

Position 0 (Alice): For $pos = 0$, all arguments become 0, so:

$$PE_{(0,0)} = \sin(0/1) = 0.000 \quad (3)$$

$$PE_{(0,1)} = \cos(0/1) = 1.000 \quad (4)$$

$$PE_{(0,2)} = \sin(0/21.54) = 0.000 \quad (5)$$

$$PE_{(0,3)} = \cos(0/21.54) = 1.000 \quad (6)$$

$$PE_{(0,4)} = \sin(0/464.16) = 0.000 \quad (7)$$

$$PE_{(0,5)} = \cos(0/464.16) = 1.000 \quad (8)$$

Result: $\mathbf{PE}_0 = [0.000, 1.000, 0.000, 1.000, 0.000, 1.000]$

Position 1 (loves): Calculate frequency scaling: $10000^{2i/6}$ for $i = 0, 1, 2$:

$$PE_{(1,0)} = \sin(1/1) = 0.841 \quad (9)$$

$$PE_{(1,1)} = \cos(1/1) = 0.540 \quad (10)$$

$$PE_{(1,2)} = \sin(1/21.54) = \sin(0.0464) = 0.046 \quad (11)$$

$$PE_{(1,3)} = \cos(1/21.54) = \cos(0.0464) = 0.999 \quad (12)$$

$$PE_{(1,4)} = \sin(1/464.16) = \sin(0.0022) = 0.002 \quad (13)$$

$$PE_{(1,5)} = \cos(1/464.16) = \cos(0.0022) = 1.000 \quad (14)$$

Result: $\mathbf{PE}_1 = [0.841, 0.540, 0.046, 0.999, 0.002, 1.000]$

Step 2: Combine with Word Embeddings

Assume word embeddings:

$$\mathbf{W}_{Alice} = [0.2, 0.5, -0.1, 0.8, 0.3, -0.2] \quad (15)$$

$$\mathbf{W}_{loves} = [0.7, -0.3, 0.4, 0.1, -0.5, 0.6] \quad (16)$$

Final embeddings: $\mathbf{E}_{final} = \mathbf{W} + \mathbf{PE}$

$$\mathbf{E}_{Alice} = [0.2, 0.5, -0.1, 0.8, 0.3, -0.2] + [0.000, 1.000, 0.000, 1.000, 0.000, 1.000] \quad (17)$$

$$= [0.200, 1.500, -0.100, 1.800, 0.300, 0.800] \quad (18)$$

$$(19)$$

$$\mathbf{E}_{loves} = [0.7, -0.3, 0.4, 0.1, -0.5, 0.6] + [0.841, 0.540, 0.046, 0.999, 0.002, 1.000] \quad (20)$$

$$= [1.541, 0.240, 0.446, 1.099, -0.498, 1.600] \quad (21)$$

Summary Table

Word	Dim 0	Dim 1	Dim 2	Dim 3	Dim 4	Dim 5
Alice (pos 0)	0.200	1.500	-0.100	1.800	0.300	0.800
loves (pos 1)	1.541	0.240	0.410	1.100	-0.500	1.600

Table 1: Final combined embeddings showing word meaning + position information

Key Takeaways

1. **Problem:** Neural networks need to understand word order
2. **Solution:** Add position-specific patterns to word embeddings
3. **Method:** Use sine/cosine functions with different frequencies
4. **Result:** Each word gets semantic meaning + position awareness
5. **Benefit:** Enables understanding of grammar, syntax, and context

Positional embeddings are the mathematical foundation that allows transformers to understand that "Alice loves Bob" and "Bob loves Alice" are fundamentally different statements, making modern language AI possible.